

一、类加载过程（类的生命周期）

一个类从被加载到 JVM 开始，到卸载出内存为止，其生命周期包括 **7 个阶段**：

加载 → 验证 → 准备 → 解析 → 初始化 → 使用 → 卸载

其中，验证、准备、解析统称为**连接（Linking）**阶段。

1. 加载（Loading）

- 做什么：
 - 通过类的全限定名获取定义此类的二进制字节流（来源可以是 class 文件、jar 包、网络、动态代理生成等）。
 - 将字节流的静态存储结构转化为方法区的运行时数据结构。
 - 在堆中生成一个 `java.lang.Class` 对象，作为方法区该类的访问入口。
- 注意：加载阶段**尚未完全完成**，可能穿插了验证阶段的部分工作。

2. 验证（Verification）

- 目的：确保字节流符合 JVM 规范，不危害 JVM 安全。
- 四个步骤：
 - 文件格式验证（魔数、版本号等）。
 - 元数据验证（类是否有父类、是否实现了抽象方法等）。
 - 字节码验证（通过数据流和控制流分析，确保方法体在运行时不会出现非法操作）。
 - 符号引用验证（解析阶段前，检查能否通过符号引用找到对应的类、字段、方法）。

3. 准备（Preparation）

- 做什么：为类的**静态变量**（`static` 字段）分配内存并设置**默认零值**。
- 注意：此时**不包含实例变量**（实例变量在对象实例化时分配）。
- 特殊情况：如果静态变量被 `final` 修饰（常量），则准备阶段直接赋值为用户定义的值（如 `static final int a = 100;` 在准备阶段就赋值为 100）。

4. 解析（Resolution）

- 做什么：将常量池中的**符号引用**替换为**直接引用**（内存地址或句柄）。
- 解析目标：类或接口、字段、类方法、接口方法、方法类型、方法句柄、调用点限定符。
- 解析时机：JVM 规范允许在类加载的解析阶段进行，也允许在首次使用时才解析（即**延迟解析**，如 `invokestatic` 指令第一次执行时）。多数 JVM 采用后者，以提升启动速度。

5. 初始化 (Initialization)

- 做什么：执行类构造器 `<clinit>()` 方法。
 - `<clinit>()` 由编译器自动收集类中所有静态变量赋值动作和静态代码块合并而成。
 - 父类的 `<clinit>()` 先于子类执行。
 - 若类没有静态块或静态变量赋值，编译器可不生成 `<clinit>()`。
- 触发初始化的条件（主动引用）：
 - `new` 对象、读写静态变量（非 `final`）、调用静态方法。
 - 反射调用 `Class.forName()`（指定 `initialize=true`）。
 - 初始化子类时，父类未初始化则先初始化父类。
 - JVM 启动时指定的主类（含 `main` 方法的类）。
 - `MethodHandle` 或 `VarHandle` 首次调用时。
 - 接口定义了 `default` 方法，其实现类初始化时需先初始化该接口。
- 不会触发初始化的情况（被动引用）：
 - 通过子类引用父类的静态字段（只初始化父类）。
 - 通过数组定义引用类（如 `MyClass[] arr = new MyClass[10]`）。
 - 引用类的 `final` 常量（常量在编译期已存入调用类的常量池，与类本身无关）。

二、类加载机制

1. 三种内置类加载器

类加载器	加载路径	作用
Bootstrap ClassLoader （启动类加载器）	<code>JAVA_HOME/jre/lib</code> 下核心库（ <code>rt.jar</code> 、 <code>resources.jar</code> 等）	加载 Java 核心类（如 <code>java.lang.*</code> ），由 C++ 实现，Java 中获取为 <code>null</code> 。
Extension ClassLoader （扩展类加载器）	<code>JAVA_HOME/jre/lib/ext</code> 或系统属性 <code>java.ext.dirs</code> 指定的目录	加载扩展类（如 <code>javax.*</code> ），父加载器为 Bootstrap。
Application ClassLoader （应用类加载器）	<code>classpath</code> （环境变量 <code>CLASSPATH</code> 或 <code>-cp</code> 指定）	加载用户类路径上的类，父加载器为 Extension。

2. 双亲委派模型 (Parents Delegation Model)

- 工作流程：

当类加载器收到类加载请求时，**不会自己立即加载**，而是先委派给父加载器。

每一层都如此，直到顶层的 Bootstrap 类加载器。

只有父加载器反馈无法加载（找不到类）时，子加载器才尝试自己加载。

- 代码体现 (`ClassLoader.loadClass` 方法逻辑):

```
protected Class<?> loadClass(String name, boolean resolve) throws
ClassNotFoundException {
    synchronized (getClassLoadingLock(name)) {
        // 1. 检查已加载的类
        Class<?> c = findLoadedClass(name);
        if (c == null) {
            try {
                if (parent != null) {
                    c = parent.loadClass(name, false); // 委派给父加载器
                } else {
                    c = findBootstrapClassOrNull(name); // 父为 null 则找 Bootstrap
                }
            } catch (ClassNotFoundException e) {
                // 父加载器找不到, 则自己尝试加载
            }
            if (c == null) {
                c = findClass(name); // 自己加载
            }
        }
        if (resolve) resolveClass(c);
        return c;
    }
}
```

- 为什么要双亲委派?
 - 避免类重复加载: 同一个类只被加载一次。
 - 核心类库安全: 防止用户自定义的 `java.lang.String` 等类替代核心类, 因为委派给 Bootstrap 加载器后, 会优先加载 `rt.jar` 中的类, 用户自定义的恶意类无法被加载。

3. 打破双亲委派模型的场景

- **SPI (Service Provider Interface):** 如 JDBC 的 `DriverManager` 需要加载应用类路径下的驱动实现, 但 `DriverManager` 由 Bootstrap 加载器加载, 无法委派到 Application 加载器。
解决方案: 使用线程上下文类加载器 (`Thread.currentThread().getContextClassLoader()`), 让父加载器调用子加载器去加载类——逆向委派。
- **热部署:** OSGi、Tomcat 的 `WebappClassLoader` 等, 为了实现模块间类隔离和动态部署, 会先自己加载, 再委派给父加载器。
- **自定义类加载器:** 重写 `loadClass` 方法, 选择性打破默认委派逻辑。

4. 自定义加载器

1. 继承 `java.lang.ClassLoader`。
2. 重写 `findClass(String name)` 方法 (推荐, 遵循双亲委派)。
 - 读取 `.class` 文件的字节数组。
 - 调用 `defineClass(String name, byte[] b, int off, int len)` 生成 `Class` 对象。

3. 如果需要完全打破双亲委派，重写 `loadClass` 方法。

```
public class MyClassLoader extends ClassLoader {
    private String classPath;

    public MyClassLoader(String classPath) {
        this.classPath = classPath;
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        byte[] data = loadClassData(name);
        return defineClass(name, data, 0, data.length);
    }

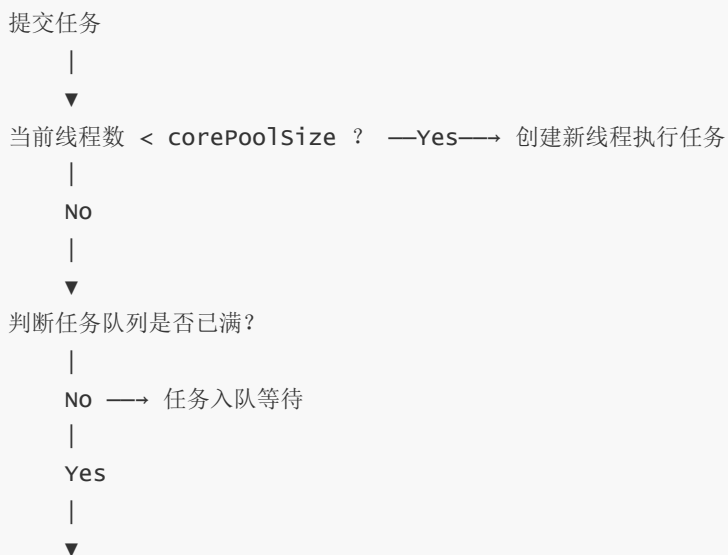
    private byte[] loadClassData(String name) {
        // 根据类名从自定义路径读取 .class 文件字节流
        // 省略实现细节
    }
}
```

三、Java程序执行流程

一个 Java 程序从源代码到最终在计算机上运行，需要经过**编译**、**类加载**、**字节码校验**、**解释执行或即时编译（JIT）**等阶段。下图概括了整个流程：

.java 源文件 → javac 编译 → .class 字节码文件 → 类加载器 → 字节码校验 → 解释器 / JIT 编译器 → 操作系统 / 硬件

四、线程池执行流程



当前线程数 < maximumPoolSize ? —Yes—→ 创建新线程执行任务

|
No
|
▼

触发拒绝策略 (RejectedExecutionHandler)

五、SpringMVC执行流程

1. 客户端发送请求 → DispatcherServlet (前端控制器) 接收。
2. DispatcherServlet 调用 HandlerMapping → 根据请求 URL 找到对应的 HandlerExecutionChain (包含处理器 Controller 和拦截器列表)。
3. DispatcherServlet 调用 HandlerAdapter → 适配并执行处理器 (Controller)。
4. Controller 执行业务逻辑 → 返回 ModelAndView 对象 (或直接返回数据, 如 @ResponseBody)。
5. DispatcherServlet 调用 ViewResolver → 解析 ModelAndView 中的逻辑视图名, 得到实际 View 对象。
6. View 渲染 → 结合 Model 数据生成响应内容 (如 HTML、JSON)。
7. 返回响应 → 将渲染结果发送给客户端。

若使用 @RestController 或 @ResponseBody, 步骤 5-6 会跳过视图解析, 直接通过 `HttpMessageConverter` 将返回对象序列化为 JSON/XML 等格式写入响应体。

客户端

|
| HTTP 请求
▼

DispatcherServlet (前端控制器)

|
| 1. 调用 HandlerMapping 获取处理器执行链
▼

HandlerMapping
根据请求 URL 找到对应的 HandlerExecutionChain
(包含 Handler 和 拦截器列表)

|
| 2. 返回 HandlerExecutionChain
▼

DispatcherServlet

|
| 3. 调用 HandlerAdapter 执行处理器
▼

HandlerAdapter



六、Spring bean的生命周期

2~7都是初始化过程

1. 实例化 (`Instantiation`)
↓
2. 属性赋值 (`Populate Properties`)
↓
3. 设置 `Bean` 名称 (`BeanNameAware`)

- ↓
4. 设置 Bean 工厂 (BeanFactoryAware)
 - ↓
 5. 设置应用上下文 (ApplicationContextAware) [可选]
 - ↓
 6. BeanPostProcessor.postProcessBeforeInitialization
 - ↓
 7. 执行自定义初始化方法:
 - @PostConstruct 标注的方法
 - InitializingBean.afterPropertiesSet()
 - 配置的 init-method
 - ↓
 8. 后置处理器 (BeanPostProcessor.postProcessAfterInitialization)
 - ↓
 - 【Bean 就绪, 可供使用】
 - ↓
 9. 销毁 (容器关闭时):
 - @PreDestroy 标注的方法
 - DisposableBean.destroy()
 - 配置的 destroy-method

七、SpringBoot启动流程

Spring Boot 的启动核心入口是 `SpringApplication.run()`，主要流程如下：

1. 创建 `SpringApplication` 实例

- 推断 Web 应用类型 (Servlet、Reactive、None)
- 加载 `META-INF/spring.factories` 中的初始化器和监听器
- 推断主类 (含 `main` 方法的类)

2. 运行 `run` 方法

- 启动计时器和事件监听器
- 准备环境 (`Environment`)：加载配置文件 (`application.properties/ym1`)、系统环境变量、命令行参数等
- 打印 Banner (可自定义或关闭)

3. 创建 `ApplicationContext` 上下文

- 根据 Web 类型创建对应的 `AnnotationConfigApplicationContext` 或 `AnnotationConfigServletWebServerApplicationContext`

4. 上下文准备阶段

- 将 `Environment` 注入上下文
- 执行 `ApplicationContextInitializer` 的 `initialize` 方法
- 打印启动日志 (Profiles、启动类等)

5. 刷新上下文 (关键)

- 调用 `refresh()` 方法 (同 Spring 注解驱动的刷新流程)

- 解析配置类、注册 Bean、自动配置（@EnableAutoConfiguration）
- 启动内嵌 Web 容器（如 Tomcat、Jetty、Undertow）
- 发布 ServletWebServerInitializedEvent 等事件

6. 执行扩展点

- 执行 CommandLineRunner 和 ApplicationRunner（按 @Order 排序）
- 发布 ApplicationReadyEvent，标记应用启动完成

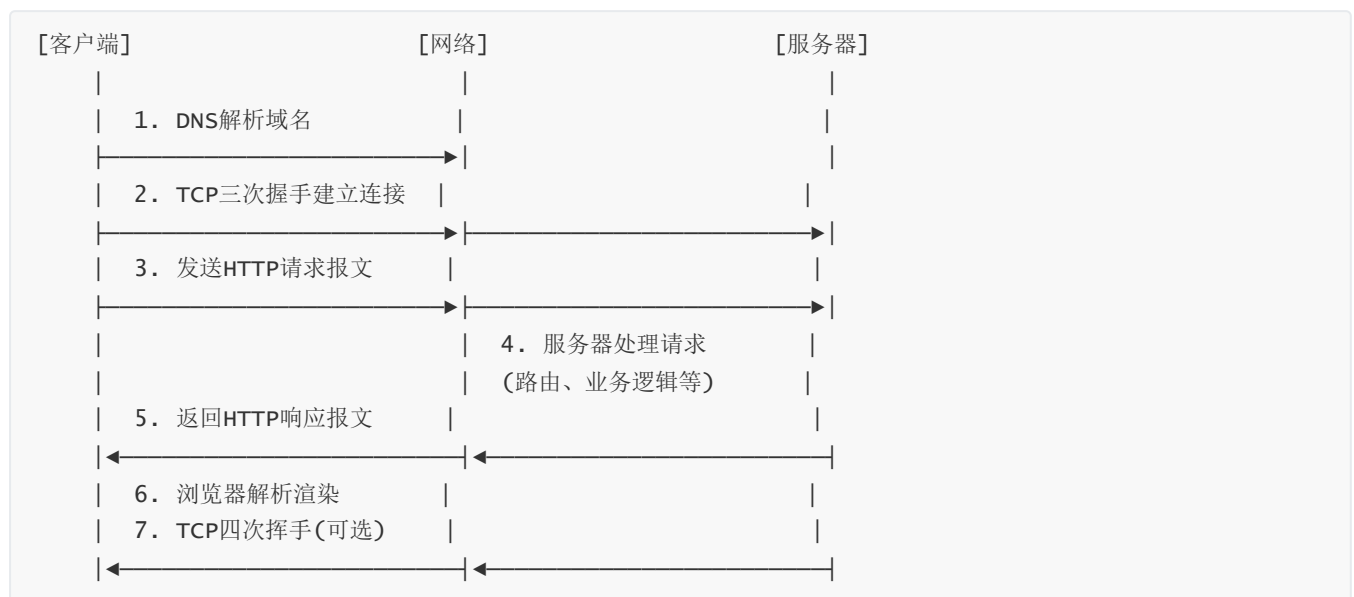
7. 等待与关闭

- 如果非 Web 应用，启动后立即退出（除非有 SpringApplication 设置）；Web 应用则阻塞等待关闭
- 注册 JVM 钩子，容器关闭时优雅释放资源

简化版：SpringApplication.run() → 加载配置 → 创建并刷新 Spring 上下文 → 启动内嵌 Web 容器 → 执行 Runner → 应用就绪。

八、HTTP通信流程

HTTP（HyperText Transfer Protocol）是一种无状态、基于请求-响应模式的应用层协议，通常运行在 TCP/IP 协议栈之上。一次完整的 HTTP 通信包含以下关键步骤：



1. 域名解析（DNS）

用户在浏览器输入 URL（如 `https://www.example.com/index.html`）后，浏览器会向 DNS 服务器查询域名对应的 IP 地址。若本地缓存中没有，则递归/迭代查询直至获得 IP。

2. 建立 TCP 连接（三次握手）

HTTP 协议基于 TCP，因此在发送 HTTP 请求前，客户端需要与服务器通过**三次握手**建立可靠连接（默认端口 80 for HTTP，443 for HTTPS）。HTTPS 还需额外进行 TLS/SSL 握手（证书验证、密钥交换）。

3. 客户端发送 HTTP 请求报文

请求报文由四部分组成：

- **请求行**：方法（GET/POST/PUT/DELETE 等）、URL、协议版本（HTTP/1.1 或 HTTP/2 等）。
- **请求头 (Header)**：键值对，如 `Host`、`User-Agent`、`Accept`、`Cookie`、`Connection` 等。
- **空行**：分隔头部和正文。
- **请求体 (Body)**：可选，如 POST 表单数据、JSON、文件等。

4. 服务器处理请求并返回响应报文

服务器接收报文后，解析请求行和头部，定位资源（静态文件或动态脚本），执行相应逻辑（如数据库查询），然后生成响应报文。

响应报文包含：

- **状态行**：协议版本、状态码（200 OK、404 Not Found、500 等）、状态描述。
- **响应头**：`Content-Type`、`Content-Length`、`Set-Cookie`、`Cache-Control` 等。
- **空行**。
- **响应体**：HTML、JSON、图片二进制数据等。

5. 浏览器解析与渲染

浏览器收到响应后，根据 `Content-Type` 决定如何处理。对于 HTML，会解析 DOM 树、CSS 树，执行 JavaScript，布局并绘制页面。如果页面中包含其他资源（CSS、JS、图片），会再次发起新的 HTTP 请求（通常并行、有限数量）。

6. 连接管理

- **HTTP/1.0**：默认短连接，每次请求后关闭 TCP 连接。
- **HTTP/1.1**：默认持久连接（`Connection: keep-alive`），一个 TCP 连接可复用发送多个请求，减少握手开销。
- **HTTP/2**：多路复用，在一个连接上并行交错发送多个请求/响应，进一步降低延迟。
- **关闭连接**：当一方发送 `Connection: close` 或空闲超时时，通过 TCP 四次挥手断开连接。

7. 关键特点

- **无状态**：服务器不保留请求之间的用户信息，需借助 Cookie、Session、Token 等机制维持会话。
- **明文传输 (HTTP)**：数据未加密，易被窃听；HTTPS 通过 TLS 加密保证机密性与完整性。
- **灵活性**：可传输任意类型数据（文本、图像、视频等），通过 `Content-Type` 标识。